Разработка безопасного программного обеспечения (Модуль бинарных уязвимостей)

Задача
- Вариант 1 (Номер студенческого билета заканчивается на 1)
- Вариант 2 (Номер студенческого билета заканчивается на 2,4,6,8)
- Вариант 3 (Номер студенческого билета заканчивается на 3,5,7,9)
- Вариант 4 (Номер студенческого билета заканчивается на 0)

Требования для лабораторной работы

Литература

Пример выполнения лабораторной работы
- ПО с уязвимостью переполнения буфера
- ПО с закрытой уязвимостью переполнения буфера

Теоретическая часть
- Стекируем
- Инструментарий атакующего
- Во всём нужно винить C
- Латание дыр
- Что было после NX
- Рандомизация
- Война без конца

/

author: Ildar Latypov

Задача

1. Необходимо провести динамическую отладку программного обеспечения с бинарной уязвимостью
2. Проанализировать код и сделать кодревью, указав слабые места.

Вариант 1 (Номер студенческого билета заканчивается на 1)

```
#include <unistd.h>
#include <sys/types.h>
#include <stdlib.h>
#include <stdio.h>

int main()
{

  int var;
  int check = 0x04030201;
  char buf[40];

  fgets(buf,45,stdin);

  printf("\n[buf]: %s\n", buf);
  printf("[check] %p\n", check);

  if ((check != 0x04030201) && (check != 0xdeadbeef))
    printf ("\nYou are on the right way!\n");

  if (check == 0xdeadbeef)
   {
     printf("Yeah dude! You win!\nOpening your shell...\n");
     setreuid(geteuid(), geteuid());
     system("/bin/bash");
     printf("Shell closed! Bye.\n");
   }
   return 0;
}
```

Вариант 2 (Номер студенческого билета заканчивается на 2,4,6,8)

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>

void shell() {
    setreuid(geteuid(), geteuid());
    system("/bin/bash");
}

void sup() {
    printf("Hey dude ! Waaaaazzaaaaaaaa ?!\n");
}

void main()
{
    int var;
    void (*func)()=sup;
    char buf[128];
    fgets(buf,133,stdin);
```

```
    func();
}
```

Вариант 3 (Номер студенческого билета заканчивается на 3,5,7,9)

```
#include <stdio.h>
#include <unistd.h>

int main(int argc, char *argv[]){
    FILE *secret = fopen("/challenge/app-systeme/ch5/.passwd", "rt");
    char buffer[32];
    fgets(buffer, sizeof(buffer), secret);
    printf(argv[1]);
    fclose(secret);
    return 0;
}
```

Вариант 4 (Номер студенческого билета заканчивается на 0)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

void    checkArg(const char *a)
{
  while (*a)
    {
      if (   (*a == ';')
          || (*a == '&')
          || (*a == '|')
          || (*a == ',')
          || (*a == '$')
          || (*a == '(')
          || (*a == ')')
          || (*a == '{')
          || (*a == '}')
          || (*a == '`')
          || (*a == '>')
          || (*a == '<') ) {
        puts("Forbidden !!!");
        exit(2);
      }
        a++;
    }
}

int     main()
{
  char  *arg = malloc(0x20);
  char  *cmd = malloc(0x400);
  setreuid(geteuid(), geteuid());

  strcpy(cmd, "/bin/ls -l ");
```

```
  printf("Enter directory you want to display : ");

  gets(arg);
  checkArg(arg);

  strcat(cmd, arg);
  system(cmd);

  return 0;
}
```

3. Предложить исправление для кода с целью избежать ошибки

Требования для лабораторной работы

1. Рекомендуется делать все в виртуальной машине Web Security Dojo или Ubuntu версии 18.04 и старше
2. Для работы можно установить Orace VM Virtualbox и расширения для него VirtualBox Extension Pack
3. Скачать последний архив с виртуальной машиной Web Security Dojo
4. Установка и настройка SonarQube с использованием Docker

Литература

1. Классический buffer overflow и создание шеллкода вручную
2. Статья Переполнение буфера и перезапись значения переменных - Изучение методов эксплуатации на примерах, часть 1
3. Переполнение буфера для чайников
4. Курс MIT «Безопасность компьютерных систем». Лекция 3: «Переполнение буфера: эксплойты и защита», часть 1
5. Курс MIT «Безопасность компьютерных систем». Лекция 3: «Переполнение буфера: эксплойты и защита», часть 2
6. https://Курс MIT «Безопасность компьютерных систем». Лекция 3: «Переполнение буфера: эксплойты и защита», часть 3
7. PVS-Studio: поиск дефектов безопасности
8. CWE-119: Improper Restriction of Operations within the Bounds of a Memory Buffer
9. CWE-787: Out-of-bounds Write
10. CWE-121: Stack-based Buffer Overflow
11. OWASP description Buffer Overflow
12. GDB Tutorial
13. Debugging with gdb
14. IDA Pro и техники реверс-инжиниринга
15. Фундаментальные основы хакерства. Мастер класс по анализу исполняемых файлов

16. Тонкости дизассемблирования (дизассемблирование в уме)
17. Как устроены дыры в безопасности: переполнение буфера

Пример выполнения лабораторной работы

ПО с уязвимостью переполнения буфера

Установим компилятор языка СИ gcc, отладчик gdb

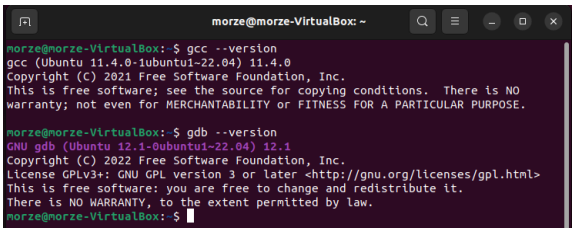


Рисунок 1 – Версия gcc и gdb

```
/* Исходный код программы с уязвимостью переполнения буфера */
#include <stdio.h>
void read_string()
{
  char buffer[16];
  printf("String: ");
  scanf("%s", buffer);
}

int main()
{
  read_string();
  return 0;
}

/* Скомпилируем исходный файл с опциями –fno-stack-protector –z execstack –m32 –g main.c –o
```

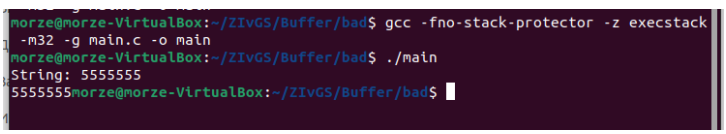
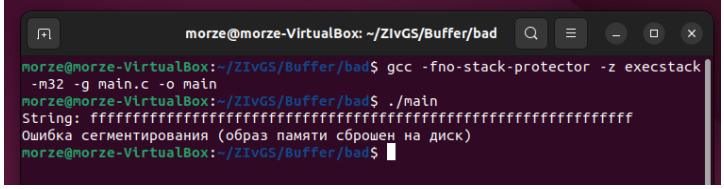


Рисунок 2 - Компиляция и запуск программы без переполнения буфера

Произведем переполнения буфера, введя строку больше буфера представлено на рисунке 3.


morze@morze-VirtualBox: ~/ZIvGS/Buffer/bad
morze@morze-VirtualBox:~/ZIvGS/Buffer/bad$ gcc -fno-stack-protector -z execstack
-m32 -g main.c -o main
morze@morze-VirtualBox:~/ZIvGS/Buffer/bad$ ./main
String: ffffff

После запуска программы байтами DCBA перезаписан адрес возврата, вместо адреса возврата появятся следующие значения 44 43 42 41. Это коды букв D C B A. Возникла ошибка segmentation fault, переход будет некорректный и возникает ошибка, представленная на рисунке 7.

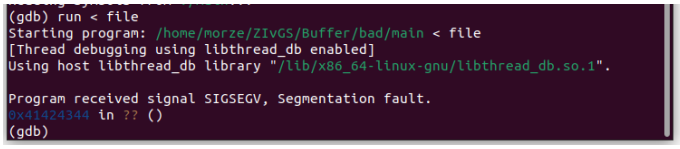


Рисунок 7 – Файл для переполнения буфера с строкой DCBA

ПО с закрытой уязвимостью переполнения буфера

Ниже представлен программный код, которая убирает уязвимость переполнения буфера.

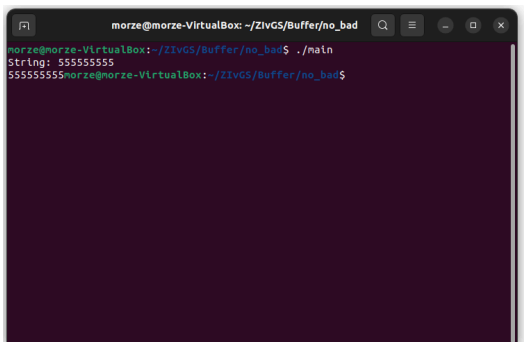
```
/* Программный код с закрытой уязвимостью переполнения буфера */

#include <stdio.h>

void read_string()
{
  int size Buffer = 16;
  char buffer[sizeBuffer];
  printf("String: ");
  fgets(buffer, sizeBuffer, stdin)
  printf("%s", buffer);
}

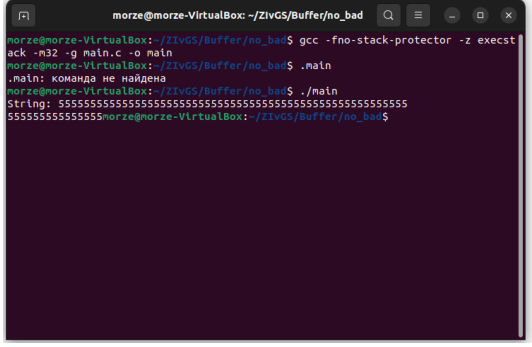
int main()
{
  read_string();
  return 0;
}
```

Скомпилируем программу и запустим её. Введем строку, которая меньше размера буфера и вывод строки правильный представлен на рисунке 8.



Рисунке 8 - Скомпилированная программа без переполнения буфера

Введём в программу строку, которая будет переполнять буфер и посмотрим на вывод программы, который представлен на рисунке 9.



Рисунке 9 - Скомпилированная программа с переполнения буфера

Как видно из вывода программы, мы обрезали лишние элементы и вывели только первые 16 символов.

Попробуем записать строку, которую мы использовали для переполнения буфера. Данная операция представлена на рисунке 10.

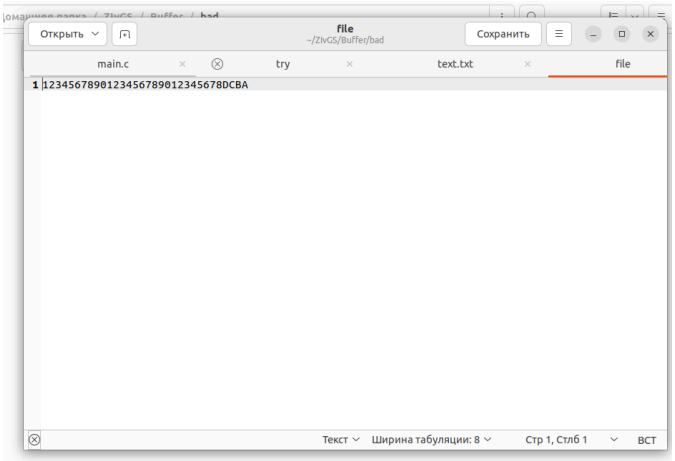


Рисунок 10 – Файл для переполнения буфера с строкой DCBA

На рисунке 11, мы видим, что программный код с закрытой уязвимостью не дает нам возможность перезаписать код возврата.

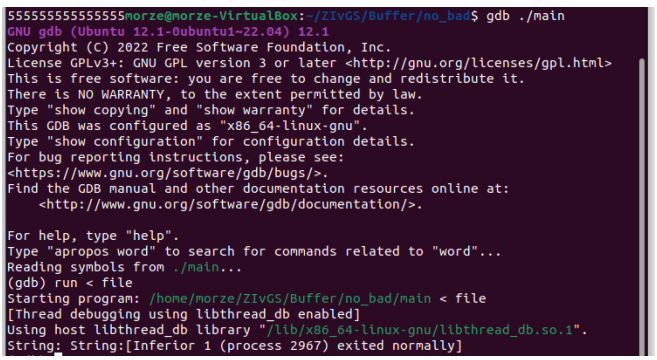


Рисунок 11 – Выполнения кода с закрытой уязвимостью, и на вход подается строка, которая вызывает переполнения буфера

Теоретическая часть

Переполнение буфера (buffer overflow) давно известно в области компьютерной безопасности. Даже первый само-распространяющийся Интернет-червь — Червь Морриса 1988 года — использовал переполнение буфера в Unix-демоне finger для распространения между машинами. Двадцать семь лет спустя, переполнение буфера остаётся источником проблем. Разработчики Windows изменили свой подход к безопасности после двух основанных на переполнении буфера эксплойтов в начале двухтысячных. А обнаруженное в мае сего года переполнение буфера в Linux драйвере (потенциально) подставляет под удар миллионы домашних и SMB маршрутизаторов.

По своей сути, переполнение буфера является невероятно простым багом, происходящим из распространённой практики. Компьютерные программы часто работают с блоками данных, читаемых с диска, из сети, или даже с клавиатуры. Для размещения этих данных, программы выделяют блоки памяти конечного размера — буферы. Переполнение буфера происходит, когда происходит запись или чтение объёма данных большего, чем вмещает буфер.

На поверхности, это выглядит как весьма глупая ошибка. В конце концов, программа знает размер буфера, а значит, должно быть несложно удостоверится, что программа никогда не попытается положить в буфер больше, чем известный размер. И вы были бы правы, рассуждая таким образом. Однако переполнения буфера продолжают происходить, а результаты часто представляют собой катастрофу для безопасности.

Чтобы понять, почему происходит переполнение буфера — и почему результаты столь плачевны — нам нужно рассмотреть то, как программы используют память, и как программисты пишут код.

(Примечание автора: мы рассмотрим, в первую очередь, переполнение стекового буфера (stack buffer overflow). Это не единственный вид переполнения, но оно является классическим и наиболее изученным видом)

Стекируем

Переполнение буфера создаёт проблемы только в нативном коде — т.е. в таких программах, которые используют набор инструкций процессора напрямую, без посредников вроде Java или Python. Переполнения связаны с тем как процессор и программы в нативном коде управляют памятью. Различные операционные системы имеют свои особенности, но все современные распространённые платформы следуют общим правилам. Чтобы понять, как работают атаки, и какие бывают способы противодействия, сначала немного рассмотрим использование памяти.

Важнейшей концепцией является адрес в памяти. Каждый отдельный байт памяти имеет соответствующий числовой адрес. Когда процессор читает или записывает данные в основную память (ОЗУ, RAM), он использует адрес памяти того места, откуда происходит считывание или куда производится запись. Системная память

используется не только для данных; она также используется для размещения исполняемого кода, из которого состоит программа. Это означает, что каждая из функций запущенной программы также имеет адрес.

Изначально, процессоры и операционные системы использовали адреса физической памяти: каждый адрес памяти напрямую соотносился с адресом конкретного куска RAM. Хотя, некоторые части современных операционных систем всё ещё используют физические адреса, все современные операционные системы используют схему, именуемую виртуальной памятью.

При использовании виртуальной памяти, прямое соответствие между адресом памяти и физическим участком RAM отсутствует. Вместо этого, программы и процессор оперируют в виртуальном пространстве адресов. Операционная система и процессор совместно поддерживают соответствие (mapping) между адресами виртуальной и физической памяти.

Такая виртуализация позволяет использовать несколько важных функций. Первая и важнейшая — это *защищённая памят*ь. Каждый отдельный процесс получает свой *собственный* набор адресов. Для 32-битного процесса, адреса начинаются с нуля (первый байт) и идут до 4,294,967,295 (в шестнадцатеричном виде, 0xffff'ffff; 2^32 — 1). Для 64-битного процесса, адреса продолжаются до 18,446,744,073,709,551,615 (0xffff'ffff'ffff'ffff, 2^64 — 1). Таким образом, у каждого процесса есть свой собственный адрес 0, за ним свой адрес 1, свой адрес 2 и так далее.

(Примечание автора: Далее в статье я буду говорить о 32-битный системах, если не указано иное. В данном аспекте разница между 32-битными и 64-битными несущественна; для ясности я буду придерживаться единой битности)

Поскольку каждый процесс получает свой собственный набор адресов, эта схема является простым способом предотвратить повреждение памяти одного процесса другим: все адреса к которым процесс может обращаться принадлежат только ему. Это гораздо проще и для самого процесса; адреса физической памяти, хотя они в широком смысле работают также (это просто номера, начинающиеся с нуля), имеют особенности, которые делают их несколько неудобными в использовании. Например, они обычно не-непрерывные; например, адрес 0x1ff8'0000 используется для памяти режима системного управления процессора — небольшой кусок памяти, недоступный обычным программам. Память PCIe-карт также находится в этом пространстве. С адресами виртуальной памяти таких неудобств нет.

Что же находится в адресном пространстве процесса? Говоря в общем, существуют четыре распространённых объекта, три из которых представляют для нас интерес. Неинтересный для нас блок, в большинстве операционных систем, — «ядро операционной системы». В интересах производительности, адресное пространство обычно разделяют на две половины, нижняя из которых используется программой, а верхняя занимается адресным пространством ядра. Половина, отданная ядру, недоступна из половины занятой программой, однако само ядро может читать память

программы. Это является одним из способов передачи данных в функции ядра.

В первую очередь разберёмся с исполняемой частью и библиотеками, составляющими программу. Главный исполняемый файл (main executable) и все его библиотеки загружаются в адресное пространство процесса, и все составляющие их функции, таким образом, имеют адрес в памяти.

Вторая часть используемой программой памяти используется для хранения обрабатываемых данных и обычно называется кучей (heap). Эта область, например, используется для хранения редактируемого документа, или просматриваемой веб-страницы (со всеми её объектами JavaScrit, CSS и т.п.), или карты игры, в которую играют.

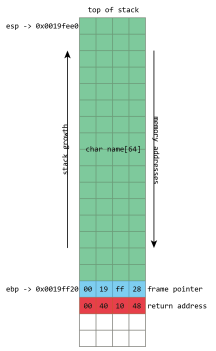
Третья и важнейшая часть — стек вызовов, обычно называемый просто стеком. Это самый сложный аспект. Каждый поток в процессе имеет свой стек. Это область памяти, используемая для одновременного отслеживания как текущей функции исполняемой в потоке, так и всех предшествующих функций — тех, что были вызваны, чтобы попасть в текущую функцию. Например, если функция *a* вызывает функцию *b*, а функция *b* вызывает функцию *c*, то стек будет содержать информацию об *a*, *b* и *c*, в таком порядке.

Стек вызовов является специализированной версией структуры данных, называемой «стеком». Стеки являются структурами переменной длины, предназначенными для хранения объектов. Новые объекты могут быть добавлены (pushed) в конец стека (обычно называемого «вершиной» стека) и объекты могут быть сняты (popped) со стека. Только вершина стека подлежит изменению с использованием push и pop, таким образом, стек устанавливает строгий порядок сортировки: объект, который последним положили в стек, будет тем, который будет снят с него следующим.

Важнейшим объектом, хранимым в стеке вызовов, является адрес возврата (return address). В большинстве случаев, когда программа вызывает функцию, эта функция выполняет то, что должна (включая вызов других функций), а затем возвращает управление в функцию, которая её вызвала. Для возврата к вызывающей функции необходимо сохранить запись о ней: исполнение должно продолжиться с инструкции следующей *после* инструкции вызова. Адрес этой инструкции называется адресом возврата. Стек используется для хранения этих адресов возврата: при каждом вызове функции, в стек помещается адрес возврата. При каждом возврате, адрес снимается со стека и процессор начинает выполнять инструкцию по этому адресу.

Стековая функциональность является настолько базовой и необходимой, что большинство, если не все процессоры имеют встроенную поддержку этих концепций. Возьмём за пример процессоры x86. Среди регистров (небольших участков памяти в процессоре, доступных инструкциям), определённых в спецификации x86, два наиболее важных — *eip* (указатель инструкции — instruction pointer), и *esp* (указатель стека — stack pointer).

ESP всегда содержит адрес вершины стека. Каждый раз когда что-то добавляется в стек, значение esp уменьшается. Каждый раз, когда что-то снимается со стека, значение esp увеличивается. Это означает, что стек растёт «вниз»; по мере добавления объектов в стек, адрес хранимый в esp становится всё меньше и меньше. Несмотря на это, область памяти, на которую указывает esp, называется «вершиной стека.



Здесь мы видим простую развёртку стека с 64-символьным буфером с именем name, за ним указатель вложенного кадра (frame pointer), потом адрес возврата. В регистре esp содержится адрес вершины, в ebp — адрес указателя кадра.

EIP содержит адрес текущей инструкции. Процессор поддерживает значение eip самостоятельно. Он читает поток инструкций из памяти и изменяет значение eip соответственно, так что он всегда содержит адрес инструкции. В рамках x86 существует инструкция для вызова функций, *call*, а также инструкция для возврата — *ret*.

CALL принимает один операнд, адрес вызываемой функции (хотя есть несколько способов передать его). Когда выполняется call, указатель стека esp уменьшается на 4 байта (32 бита), и адрес инструкции следующей за call — адрес возврата — помещается в область памяти, на которую теперь указывает esp. Другими словами, адрес возврата помещается в стек. Затем, значением eip устанавливается равным адресу, переданному в качестве операнда call, и выполнение продолжается с этой точки.

RET производит обратную операцию. Простой ret не принимает операндов. Процессор сначала считывает значение по адресу памяти, хранимому в esp, потом увеличивает esp на 4 байт — снимает адрес возврата со стека. Значение помещается в eip, и выполнение продолжается с этого адреса.

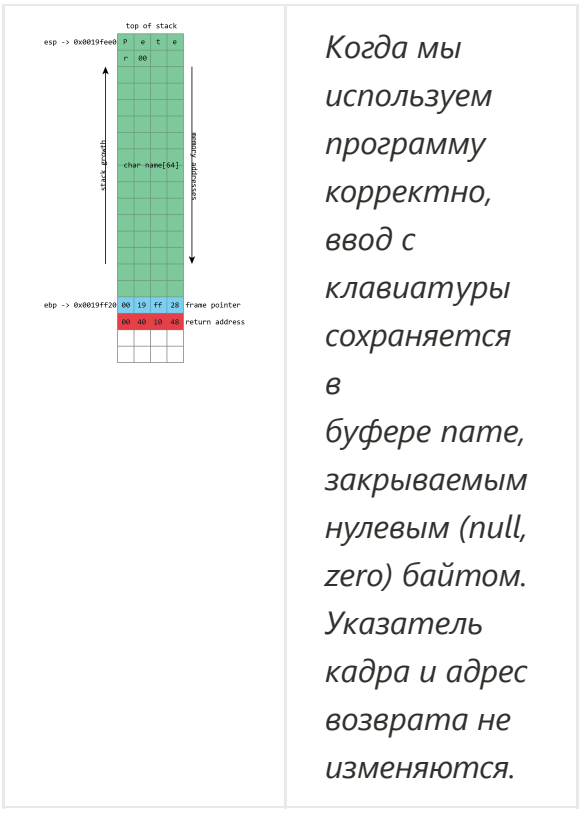
> *(Примечание переводчика: в этом месте в авторском тексте приводится видео с демонстрацией call и ret.)*

Если бы стек вызовов хранил *только* набор адресов возврата, проблемы бы не было. Реальная проблема приходит со всем остальным, что кладут в стек. Так выходит,

что стек — это быстрое и эффективное место хранения данных. Хранение данных в куче относительно сложно: программа должна отслеживать доступное в куче место, сколько занимает каждый из объектов и прочее. При этом работа со стеком проста: чтобы разместить немного данных, достаточно просто уменьшить значение указателя. А чтобы почистить за собой, достаточно увеличить значение указателя.

Это удобство делает стек логичным местом для размещения переменных, используемых функцией. Функции нужно 256 байт буфера, чтобы принять ввод пользователя? Легко, просто отнимите 256 от указателя стека — и буфер готов. В конце функции, просто прибавьте 256 к указателю, и буфер отброшен.

Однако, у такого подхода существуют ограничения. Стек не подходит для хранения очень больших объектов: общий объём доступной памяти обычно фиксирован при создании потока и, часто, составляет примерно 1МБ в объёме. Поэтому большие объекты *должны* быть помещены в кучу. Стек также не применим для объектов, которые должны существовать дольше, чем выполняется одна вызванная функция. Поскольку все размещения в стеке удаляются при выходе из функции, время жизни любого из объектов в стеке не превышает времени выполнения соответствующей функции. На объекты в куче это ограничение не распространяется, они могут существовать „вечно“.



Когда мы используем программу корректно, ввод с клавиатуры сохраняется в буфере name, закрываемым нулевым (null, zero) байтом. Указатель кадра и адрес возврата не изменяются.

Стековое хранилище используется не только для явно определяемых программистом переменных; стек также используется для хранения любых значений, нужных программе. Особенно остро это проявляется в x86. Процессоры на базе x86 не отличаются большим числом регистров (всего существует 8 целочисленных регистров, и некоторые из них, как уже упомянутые eip и esp, уже заняты), поэтому функции

редко имеют возможность хранить все необходимые им значения в регистрах. Чтобы освободить место в регистрах, и при этом сохранить значение для последующего использования, компилятор поместит значение регистра в стек. Значение позднее может быть снято с регистра и помещено обратно в регистр. В жаргоне компиляторов, процесс сохранения регистров с возможностью последующего использования называется *spilling*.

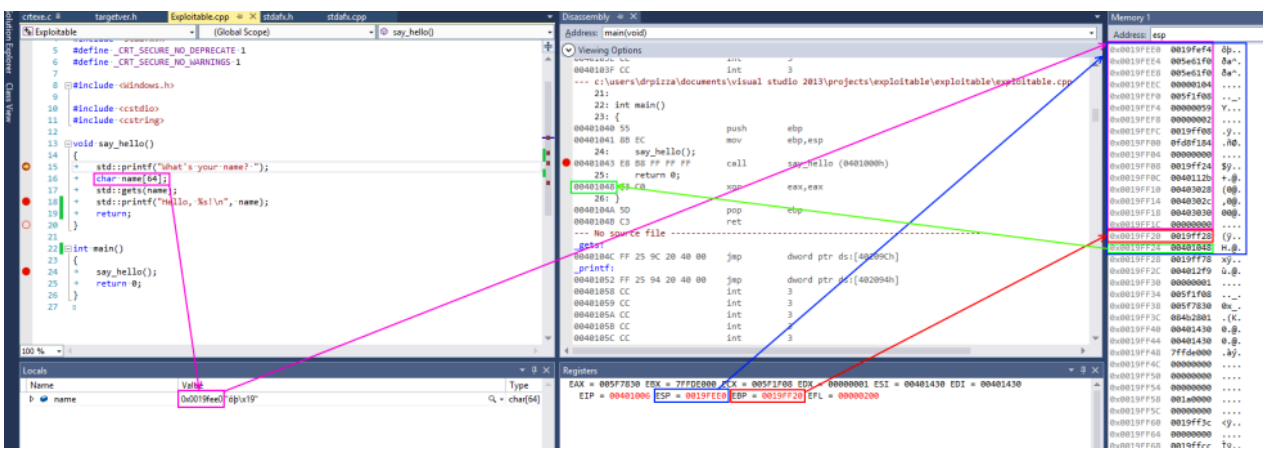
Наконец, стек часто используют для передачи аргументов функциям. Вызывающая функция помещает каждый из аргументов по очереди в стек; вызываемая функция может снять их из стека. Это не единственный способ передачи аргументов — можно использовать и регистры, например, — но это один из наиболее гибких.

Набор объектов хранимых функцией в стеке — её собственные переменные, сохранённые регистры, любые аргументы, подготавливаемые для передачи в другие функции — называются „вложенным кадром“. Поскольку данные во вложенном кадре активно используются, полезно иметь способ простой адресации к нему.

Это *возможно реализовать,* используя указатель стека, но это несколько неудобно: указатель стека всегда указывает на вершину, и его значение меняется по мере помещения и снятия объектов. Например, переменная может сначала быть расположена на позиции esp+4. После того, как ещё два значения положили в стек, и переменная стала располагаться по адресу esp+12. Если снять со стека одно из значений, переменная окажется на esp+8.

Описанное не является неподъёмной задачей, и компиляторы способны с ней справиться. Однако это делает использование указателя стека для доступа к чему-либо кроме вершины „стрёмным“, особенно при написании на ассемблере вручную.

Для упрощения задачи, обычным делом является ведение второго указателя, который хранит адрес „дна“ (т.е. начала) каждого кадра — значение, известное как указатель вложенного кадра (frame pointer). И на x86 даже есть регистр, который для этого обычно используют, *ebp*. Поскольку его значение неизменно в пределах функции, появляется способ однозначно адресовать переменные функции: значение, лежащее по адресу ebp-4, будет оставаться доступно по ebp-4 всё время жизни функции. И это полезно не только для людей — дебаггерам проще разобраться, что происходит.



Скриншот из Visual Studio демонстрирует всё это в действии на примере простой программы для x86. На процессорах x86, регистр esp содержит адрес вершины стека, в данном случае 0x0019fee0 (выделено синим). (Примечание автора: на платформе x86, стек растёт вниз, в направлении адреса памяти 0, однако эта точка всё равно сохраняет название „вершина стека"). Показанная функция хранит в стеке только переменную name, выделенную розовым цветом. Это фиксированный буфер длиной 64 байта. Поскольку это единственная переменная, её адрес тоже 0x0019fee0, такой же, как у вершины стека.

В x86 также есть регистр ebp, выделенный красным, который (обычно) выделен для хранения указателя кадра. Указатель кадра размещается сразу за переменными стека. Сразу за указателем кадра лежит адрес возврата, выделенный зелёным. Адрес возврата ссылается на фрагмент кода по адресу 0x00401048. Эта инструкция следует сразу за вызовом (call), демонстрируя то, как адрес возврата используется для продолжения исполнения там, где программа покинула вызывающую функцию.

NAME в приведённой иллюстрации относится как раз к тому роду буферов, которые регулярно переполняются. Его размер зафиксирован и составляет 64 байта. В данном случае, он заполнен набором чисел и завершается нулём. Из иллюстрации видно, что если в буфер name будет записано более 64 байт, то другие значения в стеке будут повреждены. Если записать на четыре байта больше, указатель кадра будет уничтожен. Если записать на восемь байт больше, то и указатель кадра, и адрес возврата будут перезаписаны.

Очевидно, что это ведёт к повреждению данных программы, но проблема с переполнением буфера куда серьёзнее: они ведут к выполнению [произвольного] кода. Это происходит потому, что переполненный буфер не просто перезапишет данные. Также могут оказаться перезаписаны более важные вещи, хранимые в стеке — адреса возврата. Адрес возврата контролирует то, какие инструкции процессор будет выполнять, когда закончит с текущей функцией; предполагается, что это будет какой-то адрес внутри вызывающей функции, но если это значение будет переписано переполнением буфера, оно может указывать куда угодно. Если атакующие могут контролировать переполнение буфера, то они могут контролировать и адрес возврата. Если они контролируют адрес возврата, они могут указать процессору, что делать дальше.

У процессора, скорее всего, нет красивой удобной функции „скомпрометировать машину", которую бы запустил атакующий, но это не слишком важно. Тот же буфер, который используется для изменения адреса возврата, можно использовать для хранения небольшого куска исполнимого кода (shellcode, шеллкод), который, в свою очередь, скачает вредоносный исполнимый файл, или откроет сетевое соединение, или исполнит любые другие пожелания атакующего.

Традиционно, сделать это было тривиально просто, по причине, которая у многих вызывает удивление: обычно, каждая программа будет использовать одни и те же адреса в памяти при каждом запуске, даже если вы перезагружали машину. Это означает, что позиция буфера в стеке всякий раз будет одинакова, а значит и значение, используемое для искажения адреса возврата, каждый раз будет одинаково. Атакующему достаточно лишь выяснить этот адрес однажды, и атака сработает на *любом* компьютере, исполняющем уязвимый код.

Инструментарий атакующего

В идеальном мире — с точки зрения атакующего — переписанный адрес возврата может быть просто адресом буфера. И это вполне возможно, когда программа читает данные из файла или из сети.

В других случаях, атакующий должен идти на хитрости. В функциях, обрабатывающих человеко-читаемый текст, байт с нулевым значением (null) часто имеет специальное значение; такой байт обозначает конец строки, и функции используемые для манипуляции строками — копирование, сравнение, совмещение — останавливаются. когда встречают этот символ. Это означает, что если шеллкод содержит ноль, эти процедуры его сломают.

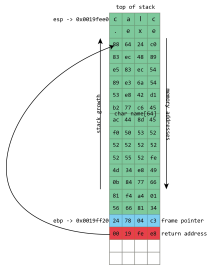
> *(Примечание переводчика: в этом месте в авторском тексте приводится видео с демонстрацией переполнения. В нём, в буфер помещают шеллкод и переписывают адрес возврата. Шеллкод запускает стандартный калькулятор Windows.)*

Чтобы это обойти, атакующий может использовать различные приёмы. Небольшие фрагменты кода для конвертирования шеллкода с нулями в эквивалентную последовательность, избегающую проблемный байт. Так возможно пролезть даже через очень строгие ограничения; например, уязвимая функция принимает на вход только данные, которые можно набрать со стандартной клавиатуры.

Собственно адрес стека часто содержит нули, и здесь есть сходная проблема: это означает, что адрес возврата нельзя записать адрес из стекового буфера. Иногда это не страшно, потому что некоторые из функций, используемых для заполнения (и, потенциально, переполнения) буферов сами пишут нули. Проявляя некоторую осторожность, их можно использовать, чтобы поместить нулевой байт точно в нужное место, устанавливая в адресе возврата адрес стека.

Но даже когда это невозможно, ситуация обходится окольными путями (indirection). Собственно программа со всеми своими библиотеками держит в памяти огромное количество исполнимого кода. Большая часть этого кода будет иметь „безопасный" адрес, т.е. не будет иметь нулей в адресе.

Тогда, атакующему нужно найти подходящий адрес, содержащий инструкцию вроде *call esp* (x86), которая использует значение указателя стека в качестве адреса функции и начинает её исполнение, чем идеально подходит для шеллкода спрятанного в стековом буфере. Атакующий использует адрес инструкции *call esp* для записи в качестве адреса возврата; процессор сделает лишний прыжок через этот адрес, но всё равно попадёт на шеллкод. Этот приём с прыжком через другой адрес называется „трамплином“.



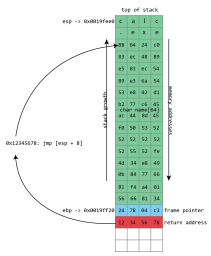
Для эксплуатации переполнения, вместо того чтобы просто забить всё некими символами, атакующий пишет в буфер шеллкод: короткий участок исполнимого кода, который выполнит некое выбранное атакующим действие. Адрес возврата переписывается соответствующим адресом из буфера, заставляя процессор исполнять шеллкод при попытке возврата из процедуры.

Это работает потому, повторюсь, что программа и её библиотеки при каждом запуске размещаются в одни и те же области памяти — даже между перезагрузками и даже на разных машинах. Одним из интересных моментов в этом деле является то, что библиотеке, от которой выполняется трамплин, самой даже не нужно использовать оператор *call esp*. Достаточно, чтобы в ней были два подходящих байта (в данном случае, со значениями *0xff* и *0xd4*) идущие друг-за-другом. Они могут быть частью какой-то иной функции, или даже просто числом; x86 не привередлива к таким вещам. Инструкции x86 могут быть очень длинными (до 15 байт!) и могут располагаться по любому адресу. Если процессор начнёт читать инструкцию с середины — со второго

байта четырёхбайтной инструкции, к примеру — результат будет интерпретирован как совсем иная, но всё же валидная, инструкция. Это обстоятельство делает нахождение полезных трамплинов достаточно простым.

Иногда, однако, атака не может установить адрес возврата *в точности* куда требуется. Несмотря на то, что расположение объектов в памяти крайне схоже, оно может слегка отличаться от машины к машине или от запуска к запуску. Например, точное расположение подверженного атаке буфера может варьироваться вверх и вниз на несколько байт, в зависимости от имени системы или её IP-адреса, или потому, что минорное обновление программы внесло незначительное изменение. Чтобы справится с этим, полезно иметь возможность указать адрес возврата который *примерно* верен, но высокая *точность* не нужна.

Это легко делается с использованием приёма, называемого „посадочной полосой" (NOP sled, букв. „сани из NOPов" (спасибо Halt за корректный русскоязычный термин — прим.пер.)). Вместо того, чтобы писать шеллкод сразу в буфер, атакующий пишет большое число инструкций NOP (означающих „no-op", т.е. отсутствие операции — говорит процессору ничего не делать), иногда сотни, перед настоящим шеллкодом. Для запуска шеллкода, атакующему нужно установить адрес возврата на позицию *где-то посреди* этих NOPов. И если мы попали в область NOPов, процессор быстро обработает их и приступит к настоящему шеллкоду.



Иногда сложно переписать адрес возврата адресом из буфера. В качестве решения, мы можем переписать адрес возврата адресом части исполнимого кода в программе-жертве (или её библиотеках). И этот фрагмент

	уже передаст управление в буфер.

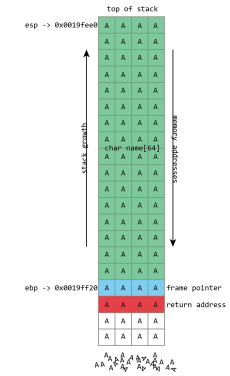
Во всём нужно винить C

Главный баг, который позволяет всё это сделать — записать в буфер больше, чем доступно места — выглядит как что-то, что легко избежать. Это преувеличение (хоть и небольшое) возлагать всю ответственность на язык программирования C, или его более или менее совместимых отпрысков, конкретно C++ и Objective C. Язык C стар, широко используем, и необходим для наших операционных систем и программ. Его дизайн отвратителен, и хотя всех этих багов можно избежать, C делает всё, чтобы подловить неосторожных.

В качестве примера враждебности C к безопасной разработке, взглянем на функцию *gets()*. Эта функция принимает один параметр — буфер — и считывает строку данных со стандартного ввода (что, обычно, означает „клавиатуру“), и помещает её в буфер. Наблюдательный читатель заметит, что функция *gets()* не включает параметр размера буфера, и как забавный факт дизайна C, отсутствует способ для функции gets() определить размер буфера самостоятельно. Это потому, что для gets() это просто не важно: функция будет читать из стандартного ввода, пока человек за клавиатурой не нажмёт клавишу Ввод; потом функция попытается запихнуть всё это в буфер, даже если этот человек ввёл много больше, чем помещается в буфер.

Это функция, которую в буквальном смысле нельзя использовать безопасно. Поскольку нет способа ограничить количество набираемого с клавиатуры текста, нет и способа предотвратить переполнение буфера функцией gets(). Создатели стандарта языка C быстро поняли проблему; версия спецификации C от 1999 года выводила gets() из обращения, а обновление от 2011 года полностью убирает её. Но её существование — и периодическое использование — показывают, какого рода ловушки готовит C своим пользователям.

Червь Морриса, первый само-распространяющийся зловред который расползся по раннему Интернету за пару дней в 1988, эксплуатировал эту функцию. Программа *fingerd* в BSD 4.3 слушает сетевой порт 79, порт *finger*. *Finger* является древней программой для Unix и соответствующим сетевым протоколом, используемым для выяснения того, кто из пользователей вошёл в удалённую систему. Есть два варианта использования: удалённую систему можно опросить и узнать всех пользователей, осуществивших вход, или можно сделать запрос о конкретном юзернейме, и программа вернёт некоторую информацию о пользователе.



К сожалению, gets() довольно глупая функция. Достаточно зажать клавишу A на клавиатуре, и она не остановится после заполнения буфера name. Она продолжит писать данные в память, перезаписывая указатель кадра, адрес возврата и всё остальное, до чего сможет дотянуться.

Каждый раз при сетевом подключении к демону finger, он начинал чтение с сети — используя gets() — в стековый буфер длиной 512 байт. При нормальной работе, fingerd затем запускал программу finger, передавая ей имя пользователя (если оно было). Программа finger выполняла реальную работу по перечислению пользователей или предоставлению информации о конкретном пользователе. Fingerd просто отвечала за сетевое соединение и запуск finger.

Учитывая, что единственный „реальный" параметр это необязательное имя пользователя, 512 байт является достаточно большим буфером. Скорее всего ни у кого нет имени пользователя и близко такой длины. Однако, нигде в системе это ограничение не было жёстким по причине использования ужасной функции gets(). Пошлите больше 512 байт по сети и fingerd переполнит буфер. И именно это сделал Роберт Моррис (Robert Morris): его эксплоит отправлял в fingerd 537 байт (536 байт данных и перевод строки, заставлявший gets() прекратить чтение), переполняя буфер и переписывая адрес возврата. Адрес возврата был установлен просто в области стекового буфера.

Исполнимая нагрузка червя Моррис была простой. Она начиналась с 400 инструкций NOP, на случай если раскладка стека будет слегка отличаться, затем короткий участок кода. Этот код вызывал шелл, */bin/sh*. Это типичный вариант атакующей нагрузки; программа fingerd запускалась под рутом, поэтому, когда при атаке она запускала шелл, шелл тоже запускался под рутом. Fingerd была подключена к сети, принимая „клавиатурный ввод" и аналогично отправляя вывод обратно в сеть. И то и другое наследовал шелл вызванный эксплойтом, и это означало, что рутовый шелл теперь был доступен атакующему удалённо.

Несмотря на то, что использования gets() легко избежать — даже во время распространения червя Морриса была доступна версия fingerd не использовавшая gets() — прочие компоненты C сложнее игнорировать, и они не менее подвержены

ошибкам. Типичной причиной проблем является обработка строк в C. Поведение, описанное ранее — останов на нулевых байтах — восходит к поведению строк в C. В языке C, строка представляет собой последовательность символов, завершаемую нулевым байтом. В C существует набор функций для работы со строками. Возможно, лучшим примером являются *strcpy()*, копирующая строку из одного места в другое, и *strcat()*, вставляющая исходную строку следом за точкой назначения. Ни одна из этих функций не имеет параметра размера буфера назначения. Обе с радостью будут бесконечно читать из источника, пока не встретят NULL, заполняя буфер назначения и беззаботно переполняя его.

Даже если строковая функция в C *имеет* параметр размера буфера, она реализует это способом, ведущим к ошибкам и переполнениям. В языке C есть пара функций родственных strcat() и strcpy(), называемых *strncat()* и *strncpy()*. Буква *n* в именах этих функций означает что они, в некотором роде, принимают размер в качестве параметра. Однако n, хотя многие наивные программисты думают иначе, не является размером буфера в который происходит запись — это число символов для считывания из источника. Если в источнике символы закончились (т.е. достигнут нулевой байт), то strncpy() и strncat() заполнят остаток нулями. Ничто в этих функциях не проверяет истинный размер назначения.

В отличии от gets(), эти функции возможно использовать безопасным образом, только это не просто. В языках C++ и Objective-C есть лучшие альтернативы этим функциям C, что делает работу со строками проще и безопаснее, однако функции C также поддерживаются в целях обратной совместимости.

Более того, они сохраняют фундаментальный недостаток языка C: буферы не знают своего размера, и язык никогда не проверяет выполняемые над буферами чтения и записи, допуская переполнение. Именно такое поведение привело к недавнему багу Heartbleed в OpenSSL. То не было переполнение, а пере*чтение*, когда код на C в составе OpenSSL пытался прочитать из буфера больше чем тот содержал, сливая информацию наружу.

Латание дыр

Конечно, человечество разработало множество языков в которых осуществляется проверка чтения и записи в буферы, что защищает от переполнения. Компилируемые языки, такие как поддерживаемый Mozilla язык Rust, защищённые среды исполнения вроде Java и .NET, и практически все скриптовые языки вроде Python, JavaScript, Lua и Perl имеют иммунитет к этой проблеме (хотя в .NET разработчики могут явным образом отключить защиту и подвергнуть себя подобному багу, но это личный выбор).

Тот факт, что переполнение буфера продолжает оставаться частью ландшафта безопасности, говорит о популярности C. Одой из причин этого, конечно, является большое количество унаследованного кода. В мире существует огромное количество кода на C, включая ядра всех основных операционных систем и популярных

библиотек, таких как OpenSSL. Даже если разработчики хотят использовать безопасный язык, вроде C#, у них могут оставаться зависимости от сторонних библиотек, написанных на C.

Производительность является другой причиной продолжающегося использования C, хотя смысл такого подхода не всегда понятен. Верно, что компилируемые C и C++ обычно выдают быстрый исполняемый код, и в некоторых случаях это действительно очень важно. Но у многих из нас процессоры большую часть времени простаивают; если бы мы могли пожертвовать, скажем, десятью процентами производительности наших браузеров, но при этом получить железную гарантию невозможности переполнения буфера — и других типичных дыр, мы может быть бы решили, что это не плохой размен. Только никто не торопится создать такой браузер.

Несмотря ни на что, C сотоварищи никуда не уходит; как и переполнение буфера.

Предпринимаются некоторые шаги по предупреждению этого рода ошибок. В ходе разработки, можно использовать специальные средства анализа исходного кода и запущенных программ, стараясь обнаружить опасные конструкции или ошибки переполнения до того, как эти баги пролезут в релиз. Новые средства, такие как AddressSanitizer и более старые, как Valgrind дают такие возможности.

Однако, эти оба этих инструмента требуют активного вмешательства разработчика, что означает, что не все программы их используют. Системные средства защиты, ставящие целью сделать переполнения буфера менее опасными, когда они случаются, могут защитить много больше различного программного обеспечения. Понимая это, разработчики операционных систем и компиляторов внедрили ряд механизмов, усложняющих эксплуатацию этих уязвимостей.

Некоторые из этих систем нацелены на усложнение конкретных атак. Один из наборов патчей для Linux делает так, что все системные библиотеки загружаются в нижние адреса таким образом, чтобы содержать, по крайней мере, один нулевой байт в своём адресе; это существенно усложняет их использование в переполнениях эксплуатирующих обработку строк в C.

Другие средства защиты действуют более обще. Во многих компиляторах имеется какой-либо род защиты стека. Определяемое на этапе исполнения значение, называемое „канарейкой” (canary) пишется в конец стека рядом с адресом возврата. В конце каждой функции, это значение проверяется перед выполнением инструкции возврата. Если значение канарейки изменилось (по причине перезаписи в ходе переполнения), программа немедленно рухнет вместо продолжения.

Возможно, важнейшим из средств защиты является механизм известный под именами W^X (»write exclusive-or execute"), DEP («data execution prevention»), NX («No Xecute»), XD («eXecute Disable»), EVP («Enhanced Virus Protection,» специфичный для AMD термин), XN («eXecute Never»), и, вероятно, другими. Здесь принцип прост. Эти системы стараются разделить память на записываемую (подходящую для буферов) и

исполнимую (подходящую для библиотек и программного кода), но не одновременно ту и другую. Таким образом, даже если атакующий может переполнить буфер и контролировать адрес возврата, процессор не будет выполнять шеллкод.

Как бы вы его не назвали, это важный механизм ещё и потому, что он не требует вложений. Этот подход использует защитные меры встроенные в процессор, поскольку это часть механизма аппаратной поддержки виртуальной памяти.

Как уже говорилось ранее, в режиме виртуальной памяти каждый процесс получает свой набор частных адресов памяти. Операционная система и процессор совместно поддерживают соотношение виртуальных адресов к *чему-то ещё*; иногда, виртуальный адрес отображается на физическую память, иногда в часть файла на диске, а иногда в никуда, просто потому что он не распределён. Это соотнесение гранулярно и обычно происходит частями размером в 4096 байт, именуемыми *страницами*.

Структуры данных, используемые для отображения, включают не только местоположение (физическая память, диск, нигде) каждой страницы; они также обычно содержат три бита, определяющие защиту страницы: доступна ли страница для чтения, записи и исполнения. С такой защитой, области памяти процесса используемые под данные, такие как стек, могут быть помечены на чтение и запись, но не быть исполнимыми.

Одним из интересных моментов NX является то, что его можно применить к существующим программам «задним числом», просто путём обновления операционной системы до той, что поддерживает защиту. Иногда программы налетают на проблемы. JIT (Just-in-time)-компиляторы, используемые в Java и .NET, генерируют исполнимый код в памяти на этапе исполнения, и поэтому требуют память, которую можно и писать и исполнять (хотя, одновременность этих свойств не требуется). Когда ещё не было NX, вы могли исполнять код из любой памяти, которую могли читать, поэтому в таких JIT-компиляторах не было проблемы с особыми буферами чтения-записи. С появлением NX, от них требуется удостовериться, что защита памяти изменена с чтение-запись на чтение-исполнение.

Потребность в чём-то вроде NX была ясна, особенно для Microsoft. В начале 2000-х, пара червей показала, что у компании были серьёзные проблемы с безопасностью кода: Code Red, инфицировавший не менее 359000 систем под управлением Windows 2000 с сервисом Microsoft IIS Web server в июле 2001, и SQL Slammer, инфицировавший более 75000 систем с Microsoft SQL Server в январе 2003. Эти случаи хорошо ударили по репутации.

Оба червя эксплуатировали стековое переполнение буфера, и, удивительно, что хотя они появились тринадцатью и пятнадцатью годами позже червя Морриса соответственно, метод эксплуатации был почти идентичен. Нагрузка эксплойта помещалась в буфер на основе стека и переписывала адрес возврата (одним небольшим отличием являлось то, что оба червя использовали метод трамплина.

Вместо прямого адреса стека, в адрес возврата прописывался адрес инструкции, которая возвращала управление в стек.).

Естественно, эти черви были более продвинуты и в других областях. Нагрузка Code Red не просто самовоспроизводилась; она производила дефейс веб-страниц и пыталась выполнять DoS-атаки. SQL Slammer нёс в себе всё необходимое для поиска новых целей для заражения и распространения по сети — всего в нескольких сотнях байт, при этом не оставляя следов на инфицированных машинах; перезагрузите машину — и его нет. Оба червя также работали в Интернете, который был многократно больше того, в котором распространился червь Морриса, и потому число заражений было сильно выше.

Однако основная проблема — легко эксплуатируемое переполнение стекового буфера — осталась прежней. Эти черви оказались в заголовках новостей и заставили многих сомневаться в возможности использовать Windows любого рода в качестве сервера, смотрящего в Интернет. Ответом Microsoft было начать всерьёз задумываться о безопасности. Windows XP Service Pack 2 была первым продуктом с установкой на безопасность. Было сделано несколько программных изменений, включая добавление программного межсетевого экрана, модификация Internet Explorer, препятствующая тихой установке тулбаров и плагинов, а также — поддержка NX.

Аппаратное обеспечение с поддержкой NX стало входить в быт где-то с 2004 года, когда Intel представила Prescott Pentium 4, поддержка со стороны операционных систем стала обыденностью со времён Windows XP SP2. В Windows 8 они решили ещё больше форсировать этот момент, отказавшись от поддержки процессоров, не умеющих NX.

Что было после NX

Несмотря на распространение поддержки NX, переполнение буфера остаётся актуальной проблемой информационной безопасности. Причиной тому является разработка ряда способов обхода NX.

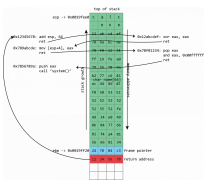
Первый из них был похож на вышеописанный трамплин, передающий контроль шеллкоду в стековом буфере через инструкцию расположенную в другой библиотеке или исполнимом файле. Вместо того чтобы искать фрагмент исполнимого кода, который бы передал управление напрямую в стек, атакующий находит фрагмент, который сам делает что-то полезное.

Возможно, лучшим кандидатом на эту роль является Unix-функция *system()*. Она принимает один параметр: адрес строки, представляющей собой команду для исполнения — и обычно этот параметр передаётся через стек. Атакующий может создать нужную команду и поместить её в переполняемый буфер, а поскольку (традиционно) расположение объектов в памяти неизменно, адрес этой строки будет

известен и может быть помещён на стек в ходе атаки. Переписанный адрес возврата в этом случае не указывает на адрес в буфере; он указывает на функцию *system()*. Когда функция подверженная переполнению завершает работу, вместо возврата в вызывающую функцию она запустит *system()*, что приведёт к исполнению заданной атакующим команды.

Вот так можно обойти NX. Функция *system()*, будучи частью системной библиотеки, уже исполнима. Эксплойту не требуется исполнять код из стека; достаточно *прочитать* команду с него. Этот приём получил название «return-to-libc» (возврат на libc, библиотеки Unix, содержащей множество ключевых функций, включая system(), и обычно загружаемой в каждый Unix-процесс, что делает её подходящей целью для такого использования) и был изобретён в 1997 году русским экспертом по информационной безопасности Solar Designer.

Хотя этот приём и полезен, у него есть ограничения. Часто функции принимают аргумент не со стека, а через регистры. Удобно передавать команды для исполнения, но они часто содержат эти дурацкие нули, что немало мешает. Кроме того, составить последовательность из различных вызовов таким способом весьма непросто. Это возможно — прописать несколько адресов возврата вместо одного — но нет способа изменить порядок следования аргументов, используя возвращаемые значения или что-либо ещё.



Вместо заполнения буфера шелкодом, мы заполняем его последовательностью адресов возврата и данными. Эти адреса возврата передают управление существующим фрагментам исполнимого кода в программе-жертве и её библиотеках. Каждый фрагмент кода выполняет операцию и выполняет возврат, передавая управление по следующему адресу возврата.

За несколько лет, return-to-libc был обобщён для обхода этих ограничений. В конце 2001 было задокументировано несколько вариантов расширения этого способа: возможность нескольких вызовов и решение проблемы нулевых байтов. Более сложный способ, решавший большую часть этих проблем, был формально описан в 2007 году: return-oriented-programming (ROP, возвратно-ориентированное программирование).

Здесь используется тот же принцип что и в return-to-libc и трамплине, но более обобщённый. Там где трамплин использует единственный фрагмент кода для передачи исполнения шеллкоду в буфере, ROP использует *много* фрагментов кода, называемых «гаджетами» в оригинальной публикации. Каждый гаджет следует определённому шаблону: он выполняет некую операцию (запись значения в регистр, запись в память, сложение регистров, и т.п.), за которой следует команда возврата. То самое свойство, что делает x86 пригодным для трамплина работает и здесь; системные библиотеки, загруженные в память процессом, содержат *сотни* последовательностей которые можно интерпретировать как «действие и возврат», а значит, могут быть использованы для ROP-атак.

Для объединения гаджетов в одно целое используется длинная последовательность адресов возврата (а также любых полезных и необходимых данных) записанных в стек в ходе переполнения буфера. Инструкции возврата прыгают с гаджета на гаджет, в то время как процессор редко (или никогда) *вызывает* функции, а только *возвращается* из них. Интересно то, что по крайней мере на x86, число и разнообразие полезных гаджетов таково, что атакующий в прямом смысле может делать *всё что угодно*; это подмножество x86, используемое особым образом, зачастую является Тьюринг-полным (хотя полный спектр возможностей будет зависеть от загружаемых программой библиотек, и следственно перечнем доступных гаджетов).

Как и в случае с return-to-libc, весь действительно исполнимый код берётся из системных библиотек, и как следствие защита вроде NX бесполезна. Большая гибкость этого подхода означает, что эксплойты могут делать то, что сложно организовать последовательностью return-to-libc, например, вызывая функции принимающие аргументы через регистры, использовать возвращаемые значения одних функций в других и прочее.

Нагрузка в ROP-атаках бывает разной. Иногда это простой код для получения шелла (доступа к командному интерпретатору). Другим распространённым вариантом является использование ROP для вызова системной функции для изменения NX-параметров страницы памяти, меняя их с записываемых на исполнимые. Сделав это, атакующий может использовать обычную, не-ROP нагрузку.

Рандомизация

Эта слабость NX давно известна, и эксплойты такого типа шаблоны: атакующий заранее знает адрес стека и системных библиотек в памяти. Всё зиждется на этом

знании, а потому очевидным решением является лишить атакующего этого знания. Именно этим занимается ASLR (Address Space Layout Randomization, Рандомизация развёртки адресного пространства): он делает случайной позицию стека и расположение в памяти библиотек и исполнимого кода. Обычно они меняются при каждом запуске программы, перезагрузке или некоторой их комбинации.

Данное обстоятельство значительным образом осложняет эксплуатацию, поскольку, совершенно неожиданно, атакующий не знает где лежат нужные для ROP фрагменты инструкций, или хотябы где находится переполняемый стек.

ASLR во многом сопутствует NX, закрывая такие крупные дыры как возврат к libc или ROP. К несчастью, он несколько менее прозрачен, чем NX. Не считая JIT-компиляторов и ряда других специфичных случаев, NX может быть безопасно внедрён в существующие программы. ASLR более проблематичен: с ним программы и библиотеки не могут полагаться в своей работе на значение адреса, в который они загружены.

В Windows, например, это не должно быть большой проблемой для DLL. В Windows, DLL всегда поддерживали загрузку в разные адреса, а вот для EXE это может быть проблемой. До ASLR, EXE всегда загружались в адрес *0x0040000* и могли полагаться на этот факт. С внедрением ASLR это уже не так. Чтобы предотвратить возможные проблемы, Windows по умолчанию требует от программ явного указания поддержки ASLR. Люди, думающие о безопасности, могут, однако, изменить это поведение по умолчанию, заставив Windows включить ASLR для всех программ и библиотек. Это почти никогда не вызывает проблем.

Ситуация вероятно хуже в Linux на x86, поскольку подход к реализации ASLR на этой платформе даёт потерю производительности до 26 процентов. Более того, этот подход *требует* компиляции программ и библиотек с поддержкой ASLR. Нет способа администратору сделать ASLR принудительным, как в Windowsю (на x64 потеря производительности пусть и не уходит совсем, но значительно снижается)

Когда ASLR активен, он даёт сильную защиту от простого взлома. Однако, он не совершенен. Например, одним из ограничений является степень случайности, которую можно получить, что особенно заметно на 32-битных системах. Хотя в адресном пространстве 4 миллиарда различных адресов, не все они доступны для загрузки библиотек или размещения стека.

Для этого существует множество ограничений. Некоторые из них состоят в широте целей. В общем случае, операционная система предпочитает загружать библиотеки близко друг-к-другу на одном из концов адресного пространства процесса, чтобы как можно больше непрерывного места было доступно приложению. Вам не хочется получить по одной библиотеке через каждые 256МБ памяти, поскольку тогда возможное наибольшее унитарное выделение памяти будет меньше 256МБ, что ограничивает возможность приложения работать с данными большого объёма.

Исполнимые файлы и библиотеки обычно должны быть загружены так чтобы начинаться, по крайней мере, на границе страницы. Обычно, это означает, что они должны быть загружены в адрес, делимый на 4096. Различные платформы могут и иметь подобные ограничения для стека; Linux, например, начианет стек на адресе делимом на 16. Системы с ограничением по памяти иногда вынуждены ещё более ограничить случайность, чтобы иметь возможность всё разместить.

Результаты бывают различными, но иногда атакующий могут угадать нужный адрес, с высокой вероятностью попадания. Даже невысокого шанса — скажем, один из 256 — может быть достаточно в некоторых ситуациях. Когда атакуешь веб-сервер, который автоматически перезапустит рухнувший процесс, не важно, что 255 из 256 атак приведут к краху процесса. Он будет перезапущен, и можно попробовать снова.

Но на 64-битных системах, адресное пространство так велико, что такой подход не поможет. У атакующего может быть только один шанс из миллиона или даже один из миллиарда, и это достаточно мало, чтобы это не было важно.

Угадывание и падение не слишком хорошая стратегия для атаки, скажем, браузеров; ни один пользователь не будет перезапускать браузер 256 раз кряду лишь бы дать атакующему шанс. В результате, эксплуатация такой уязвимости в системе с активными NX и ASLR не может быть произведена без посторонней помощи.

Такая помощь может быть нескольких видов. В браузере можно использовать JavaScript или Flash — и то и другое содержит JIT-компиляторы генерирующие исполнимый код — для заполнения памяти аккуратно сконструированным исполнимым кодом. Это создаёт что-то вроде большой посадочной полосы, приём под названием «heap spraying» («напыление кучи»). Другим подходом может быть нахождение вторичного бага, позволяющего раскрыть адреса библиотек или стека в памяти, давая атакующему достаточно информации для создания специфичного набора возвратных адресов для ROP.

Третий подход также был популярен в браузерах: использовать библиотеки, не умеющие ASLR. Старые версии, например, плагинов Adobe PDF или Microsoft Office не поддерживали ASLR, и Windows по умолчанию не форсирует ASLR. Если атакующий может вызвать загрузку такой библиотеки (например, загрузив PDF в скрытом фрейме браузера), то об ASLR можно уже не беспокоиться, а использовать эту библиотеку для целей ROP.

Война без конца

Между теми, кто эксплуатирует уязвимости и теми, кто защищает, идёт постоянная гонка вооружений. Мощные защитные системы, вроде ASLR и NX, поднимают планку, усложняя использование недостатков, и благодаря им мы оставили времена простого переполнения буфера позади, но умные атакующие могут найти комбинацию дыр и обойти эти защитные меры.

Эскалация продолжается. Набор Microsoft EMET («Enhanced Mitigation Experience Toolkit», «расширенный набор инструментов противодействия») включает ряд полу-экспериментальных средств защиты, которые могут обнаруживать heap spraying или попытки вызова определённых критичных функций в ROP-эксплойтах. Но в непрерывной цифровой войне, даже часть этих приёмов уже побеждена. Это не делает их бесполезными — сложность (а значит и цена) эксплуатации уязвимостей возрастает с каждым применённым средством противодействия — но это напоминание о необходимости постоянной бдительности.